UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Answers are unofficial reference answers, not official UTM marking schemes.

2. Marking points show the expected answer and mark allocation; check them against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Approved real questions only; no generators, placeholders or parametric variants. Questions and answers are for local study use only.

5. Structure derived from the supplied CSC429 papers; every question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 10 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION 1 (5 marks)

Source: CSC429-2024-01, page [2]

Unofficial model answer:

a) The two prominent first-generation machines inspired by John von Neumann's stored-program concept were the EDVAC (Electronic Discrete Variable Automatic Computer) and the EDSAC (Electronic Delay Storage Automatic Calculator). Other acceptable examples include the IAS machine, UNIVAC I, and Manchester Baby, but EDVAC and EDSAC are the standard textbook answers.

b) The machine developed by John von Neumann himself is the IAS machine (Institute for Advanced Study machine), also known as the von Neumann machine.

c) The IAS machine was designed to implement the stored-program concept: both instructions and data are stored in the same main memory, fetched and executed sequentially by a control unit, with an arithmetic-logic unit performing operations and input/output handled by separate devices. This architecture—now called the von Neumann architecture—became the basis for virtually all subsequent general-purpose computers.

Marking points:

- *1 mark: EDVAC (or EDSAC) named as one machine inspired by von Neumann*
- *1 mark: The other of EDVAC/EDSAC (or another valid first-generation stored-program machine) named*
- *1 mark: IAS machine (or von Neumann machine) identified as the machine developed by von Neumann*
- *2 marks: Explanation of the stored-program concept (instructions and data in same memory, sequential execution, control unit fetch-decode-execute, ALU, I/O) — 1 mark for basic idea, 1 mark for detail*

Model confidence: high.

Model note: The question does not specify a particular textbook; EDVAC and EDSAC are the most commonly cited first-generation von Neumann-inspired machines. The IAS machine is often called the 'von Neumann machine'. Some sources may also accept the Manchester Baby or UNIVAC I as inspired machines, but the marking scheme should allow reasonable alternatives.

QUESTION 2 (5 marks)

Source: CSC429-2024-01, page [2]

Unofficial model answer:

a) The fourth computer generation (approximately 1971–1980) was defined by two major technologies: (1) the microprocessor (very-large-scale integration, VLSI, which packed thousands of transistors onto a single silicon chip) and (2) the development of personal computers and user-friendly operating systems built around these microprocessors. The microprocessor made computers dramatically smaller, cheaper, faster and more energy-efficient, enabling the desktop PC and embedded computing; this directly underpins the modern computer, where every laptop, smartphone and embedded controller contains one or more microprocessors. The accompanying PC/OS technology (e.g., the Intel 4004/8080, the Altair 8800, the Apple II, and later the IBM PC) brought computing to individuals and small businesses, establishing the mass-market personal-computer industry and the software ecosystem that modern computers still rely on. (2) Another key fourth-generation technology was the introduction of high-level programming languages and standardized operating systems for microcomputers (e.g., BASIC, CP/M, MS-DOS), which made computers accessible to non-specialists and created the layered software/hardware model used in modern systems. Together these technologies transformed computers from room-sized mainframes into affordable, ubiquitous personal tools and laid the foundation for today's networked, general-purpose computing.

b) One everyday example of artificial intelligence is the voice assistant on a smartphone (e.g., Siri, Google Assistant, or Alexa), which uses speech recognition and natural-language processing to understand spoken

© Hak Cipta Universiti Teknologi MARA

commands and answer questions or control devices. Another common example is the recommendation system used by streaming services and online shops, which learns from user behaviour to suggest content or products.

Marking points:

• 1 mark: correctly identifies the microprocessor / VLSI as a fourth-generation technology

• 1 mark: explains the impact of the microprocessor (smaller, cheaper, faster computers; enabled PCs and embedded systems; basis of modern computers)

• 1 mark: identifies a second major fourth-generation technology (e.g., personal computers, high-level languages, standardized microcomputer operating systems) and explains its impact on modern computing

• 1 mark: gives one valid everyday AI example (e.g., voice assistant, recommendation system, spam filter, face unlock)

• 1 mark: briefly explains how the AI example works or is used in daily life

Model confidence: medium.

Model note: The question does not specify which two technologies are expected; the answer selects microprocessor/VLSI and PC/OS or high-level languages, which are standard textbook fourth-generation technologies. If the official scheme expects a different pair (e.g., VLSI and networking), the first two marking points may need adjustment. No figure or table was supplied.

QUESTION 3 (5 marks)

Source: CSC429-2018-12, page [2]

Unofficial model answer:

a) A byte is a unit of digital information that consists of 8 bits. It is the smallest addressable unit of memory in most computer architectures and can represent 256 distinct values (2^8), commonly used to store a single character or a small integer.

b) Base 10 is not suitable for computer number representation because digital computers use binary logic, where each bit has two stable states (0 and 1). Implementing ten distinct voltage levels for decimal digits would be complex, error-prone, and costly. Binary representation maps directly to the two-state nature of electronic components, enabling reliable storage and processing.

c) In IEEE floating point representation, the exponent field uses a biased representation. For single precision, the bias is 127, and the exponent field is 8 bits. The actual exponent value is obtained by subtracting the bias from the stored value. The maximum stored exponent value is 255, but 255 is reserved for special values (infinity and NaN). The maximum normal exponent stored is 254, which gives an actual exponent of 254 - 127 = 127. Thus, the maximum exponent value is 127, not 128, because the value 128 would require a stored exponent of 255, which is reserved.

Marking points:

• 1 mark: Definition of byte as 8 bits (or 8 bits grouped as a unit).

• 1 mark: Explanation that computers use binary (two-state) logic, making base 10 impractical.

• 1 mark: Mention of complexity/cost/reliability issues with ten-state devices.

• 1 mark: Identification of exponent bias (127) and reserved exponent value (255) for special cases.

• 1 mark: Calculation showing max normal exponent = 254 - 127 = 127, and why 128 would require reserved value 255.

Model confidence: high.

Model note: The answer assumes single-precision IEEE 754 format. The question does not specify precision, but the value 127 is specific to single precision. For double precision, the bias is 1023 and max exponent is 1023, so the question implicitly refers to single precision.

QUESTION 4 (5 marks)

Source: CSC429-2019-06, page [2]

Unofficial model answer:

a) Component X is the integrated circuit (IC), also known as a microchip/chip — a complete electronic semiconductor circuit fabricated on a piece of silicon.

b) The integrated circuit was introduced in the third computer generation (roughly 1964–1971), the generation characterised by ICs replacing discrete transistors.

c) The integrated circuit was invented by Jack Kilby (Texas Instruments, 1958), with Robert Noyce (Fairchild) independently developing a practical silicon IC shortly after; Jack Kilby is the conventional answer.

d) Input device: keyboard (or mouse). Output device: monitor (or printer). These were introduced/used in the third generation alongside ICs.

Marking points:

- *1 mark (a): integrated circuit / IC / chip*
- *1 mark (b): third generation*
- *1 mark (c): Jack Kilby (accept Robert Noyce)*
- *1 mark (d): one correct input device (keyboard/mouse)*
- *1 mark (d): one correct output device (monitor/printer)*

Model confidence: high.

Model note: Figure 1 is only supplied by caption; answer assumes the standard 'chip/IC' identification. Some syllabi credit Noyce equally for the IC invention.

QUESTION 5 (10 marks)

Source: CSC429-2024-01, page [5]

Unofficial model answer:

a) MIMD with distributed memory: The architecture consists of N independent processing nodes. Each node has its own Control Unit (CU), its own Processing Unit (PU), and its own Local Memory (LM). The CUs are not synchronized by a single global clock/instruction stream; each CU fetches and decodes instructions independently for its PU. The LMs are private to each node and are not directly addressable by other nodes. Nodes are connected by an interconnection network (e.g., a mesh, hypercube, or network switch) so that PUs can exchange messages/data explicitly. A suitable drawing shows a row/column of identical nodes, each drawn as a box containing CU, PU, and LM, with the PU connected to its LM and to the interconnection network; the network links the nodes. No single shared memory block is drawn. (Figure limitation: the original question asks for a drawing; this text describes the required diagram because no figure can be rendered in JSON.)

b) MIMD with shared memory: The architecture consists of N Processing Units (PUs), each with its own Control Unit (CU), so each PU executes its own instruction stream independently (multiple instruction streams). All PUs are connected through an interconnection network (e.g., a shared bus or crossbar switch) to one global shared memory. The shared memory is directly addressable by every PU, so communication between processors occurs by writing to and reading from shared memory locations. A suitable drawing shows several CU+PU pairs, each CU feeding its PU, all PUs attached to a common interconnection network, and that network attached to a single shared-memory block. (Figure limitation: same as above; the diagram is described rather than drawn.)

c) Virus attack on memory: i) In MIMD with distributed memory, a virus that infects one node's Local Memory can corrupt only that node's local data and programs (and possibly the node's CU/PU if the virus is executed). Because each LM is private and there is no globally shared address space, the infection does not directly spread through memory to other nodes. However, the infected node can send corrupted messages or infected files over the interconnection network to other nodes, so the virus may spread indirectly through communication; the blast radius is initially limited to one node but can grow via message passing. ii) In MIMD with shared memory, a virus that attacks the shared memory can corrupt data or code in the single global

© Hak Cipta Universiti Teknologi MARA

address space that all PUs access. Because every PU reads and writes the same shared memory, the corruption is immediately visible to all processors and can affect all running programs; the virus can spread rapidly across the whole system through shared memory locations. Recovery is harder because there is no isolated private copy per node, and one infected PU can write malicious code into shared memory for other PUs to execute.

d) One example of MIMD with distributed memory: a cluster of networked workstation PCs (or a Beowulf cluster) in which each computer has its own CPU, local RAM, and operating system, and the computers communicate by message passing over a network. (Other acceptable examples: a distributed-memory multicomputer such as Intel Paragon, IBM SP2, or a modern HPC cluster with InfiniBand.)

Marking points:

• a) 3 marks: 1 mark for showing multiple nodes each with its own CU, PU, and LM; 1 mark for correct CU-PU-LM connections within each node; 1 mark for an interconnection network linking the nodes and no shared global memory.

• b) 3 marks: 1 mark for multiple CU+PU pairs (multiple instruction streams); 1 mark for all PUs connected to a common interconnection network; 1 mark for a single shared memory block accessible by all PUs.

• c) i) 1.5 marks: 1 mark for stating that only the infected node's local memory is directly corrupted / limited direct spread; 0.5 mark for noting possible indirect spread via messages over the network.

• c) ii) 1.5 marks: 1 mark for stating that shared memory corruption affects all PUs / the whole system; 0.5 mark for noting rapid spread and difficulty of isolation because all processors share the same address space.

• d) 1 mark: one correct example of a distributed-memory MIMD system (e.g., cluster of workstations/PCs with message passing, Beowulf cluster, or a named distributed-memory multicomputer).

Model confidence: high.

Model note: The question asks for drawings in parts (a) and (b); because the response format is JSON text, the diagrams are described in words rather than rendered. The marking allocation is an unofficial suggested split of the 10 marks (3+3+3+1) and is not an official UTM scheme. Part (c) is answered conceptually because the exact virus behaviour is not specified in the question.

QUESTION 6 (10 marks)

Source: CSC429-2014-12, page [5]

Unofficial model answer:

Multiple-processor architectures are commonly classified by how processors share memory and how they are interconnected. Four standard classifications, each with an example, are: (1) Shared-memory multiprocessor (tightly coupled / UMA): all processors access a single global physical memory through a common bus or crossbar, so inter-processor communication is via shared variables in memory; example: a dual-socket Intel Xeon server board with two CPUs sharing main memory. (2) Message-passing multicomputer (loosely coupled / distributed memory): each processor has its own private local memory and processors communicate by explicitly sending and receiving messages over an interconnection network; example: a Beowulf cluster of PCs connected by Ethernet, or an IBM Blue Gene supercomputer. (3) SIMD (Single Instruction, Multiple Data) array processor: one control unit issues a single instruction that operates simultaneously on many processing elements, each working on a different data element; example: a GPU's SIMT/vector units or a classic array processor such as the ILLIAC IV. (4) NUMA (Non-Uniform Memory Access) multiprocessor: a shared address space is physically distributed among nodes, so a processor's access time depends on whether the data is in its local node's memory or a remote node's memory; example: an SGI Origin or a modern AMD EPYC multi-socket server. (Acceptable alternatives: MIMD vs SIMD vs MISD vs SISD Flynn classification; master-slave vs symmetric multiprocessing; UMA vs NUMA vs COMA; vector processors; systolic arrays.)

Marking points:

• 1 mark: correctly naming/defining classification 1 (e.g., shared-memory UMA multiprocessor) with a valid example

• 1 mark: correctly naming/defining classification 2 (e.g., message-passing distributed-memory multicomputer) with a valid example

• 1 mark: correctly naming/defining classification 3 (e.g., SIMD array/vector processor) with a valid example

• 1 mark: correctly naming/defining classification 4 (e.g., NUMA multiprocessor) with a valid example

• 2 marks: accuracy and appropriate use of terminology (tightly/loosely coupled, UMA/NUMA, SIMD, message passing, interconnection network) across the four descriptions

• 2 marks: examples are technically correct and clearly matched to each classification

• 2 marks: overall coherence, clarity and completeness of the four descriptions (each classification explained, not merely named)

Model confidence: high.

Model note: The question text does not specify a particular taxonomy (e.g., Flynn's or Tanenbaum's), so the answer uses the common memory-coupling classification and notes Flynn-style alternatives as acceptable. No figure or table was supplied; if the original paper expected a specific four-way scheme, marks should be mapped to that scheme. This is an unofficial model answer, not an official UTM marking scheme.

QUESTION 7 (10 marks)

Source: CSC429-2024-07, page [3]

Unofficial model answer:

a) The size of the word is 32 bits. Justification: In a computer system, the word size is typically defined by the width of the data bus, because the data bus determines how many bits can be transferred between the CPU and memory or I/O in a single bus transaction. Since the data bus is 32 bits, the machine word is 32 bits. (Note: The address bus width determines the addressable memory space, not the word size; the control bus width determines the number of control signals, not the word size.)

b) The size of the register is 32 bits. Justification: General-purpose registers in the CPU are designed to hold one word of data, so their size matches the word size, which is determined by the data bus width. With a 32-bit data bus, the CPU registers are 32 bits wide to accommodate arithmetic, logic, and data movement operations on full words. (Special-purpose registers such as the program counter or memory address register may be 8 bits to match the address bus, but the question asks for 'the register' generically, which conventionally refers to the data register size.)

c) If system interconnection (the bus) is blocked, the machine cycle cannot proceed normally. The machine cycle consists of fetch, decode, execute, and (if needed) memory/writeback stages. During the fetch stage, the CPU must place an address on the address bus and read the instruction from memory via the data bus; if the interconnection is blocked, the instruction fetch cannot complete, so the CPU stalls or times out. Similarly, during the execute stage, any memory read/write or I/O operation requires the bus; blocking the interconnection prevents data transfer, causing the machine cycle to halt, potentially triggering a bus error or timeout exception. Overall, the machine cycle is disrupted at the first bus-dependent step, and the system cannot make progress.

d) CPU interconnection (the system bus and its associated arbitration and control logic) provides the communication pathway between the CPU, memory, and I/O devices. Its roles include: (1) transferring addresses from the CPU to memory/I/O to select a location; (2) transferring data between the CPU and memory/I/O; (3) carrying control signals (read/write, interrupt, clock, etc.) to coordinate operations; (4) arbitrating bus access when multiple masters request the bus; and (5) enabling the CPU to fetch instructions and operands and to store results, thereby supporting the fetch-decode-execute machine cycle. Without CPU interconnection, the CPU cannot communicate with other system components, and the computer cannot function.

Marking points:

• 1 mark: Correctly state word size = 32 bits.

• 1 mark: Justify word size by data bus width (data bus determines bits transferred per transaction).

• 1 mark: Correctly state register size = 32 bits.

• 1 mark: Justify register size by word size / data bus width (registers hold one word).

© Hak Cipta Universiti Teknologi MARA

• 1 mark: Explain that blocking system interconnection prevents instruction fetch (address/ data transfer).

• 1 mark: Explain that machine cycle stalls/halts or triggers bus error/timeout due to blocked bus.

• 1 mark: Describe CPU interconnection as communication path between CPU, memory, and I/O.

• 1 mark: Mention address transfer role of interconnection.

• 1 mark: Mention data transfer role of interconnection.

• 1 mark: Mention control signal transfer and/or arbitration role of interconnection.

Model confidence: high.

Model note: Assumes 'register' refers to general-purpose data registers; special-purpose registers (e.g., PC, MAR) may be address-bus width (8 bits). The question does not specify which register, so the answer follows the common convention that register size equals word size. No figure or table was provided; answer is based solely on given bus widths.

QUESTION 8 (10 marks)

Source: CSC429-2018-06, page [3]

Unofficial model answer:

a) Binary representation is used as machine language because digital hardware is built from two-state devices (transistors that are either ON or OFF, representing 1 and 0). These two states map directly to binary digits, giving high noise immunity and simple, reliable circuit design. Binary also allows logic gates and Boolean algebra to implement arithmetic and control directly, and it is the natural representation for storing and manipulating data in registers and memory. Using only two symbols simplifies fabrication, minimises error rates, and makes operations such as addition and subtraction easy to implement with combinational logic.

b) First convert to decimal: ACEh = 10×256 + 12×16 + 14 = 2560 + 192 + 14 = 2766; BFh = 11×16 + 15 = 176 + 15 = 191. Difference = 2766 − 191 = 2575.

i) Hexadecimal: ACEh − BFh = A0Fh (borrow calculation: E−F requires borrow, giving 15−15=0 with borrow; C becomes B, B−B=0; A unchanged → A0Fh).

ii) Octal: Convert A0Fh to octal. A0Fh = 1010 0000 1111_2 = 101 000 001 111_2 = 5017_8. Check: 5×512 + 0×64 + 1×8 + 7 = 2560 + 0 + 8 + 7 = 2575. So result is 5017_8.

iii) Binary: A0Fh = 1010 0000 1111_2. So result is 101000001111_2.

Marking points:

• 1 mark: binary uses two-state devices (ON/OFF, high/low) matching 1 and 0

• 1 mark: simple, reliable, noise-immune circuit implementation

• 1 mark: direct mapping to logic gates/Boolean algebra and easy arithmetic

• 1 mark: correct decimal conversion of ACEh (2766) and BFh (191)

• 1 mark: correct hexadecimal subtraction ACEh − BFh = A0Fh

• 1 mark: correct octal result 5017_8 (with working or conversion shown)

• 1 mark: correct binary result 101000001111_2

• 1 mark: correct borrow handling in hexadecimal subtraction

• 1 mark: correct conversion between bases (hex to binary/octal)

• 1 mark: final answers clearly stated for all three parts

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. No figure or table was supplied. Marks are allocated as a suggested distribution across the 10-mark total.

QUESTION 9 (10 marks)

Source: CSC429-2014-12, page [4]

Unofficial model answer:

a) Two types of keyboard characters: (1) Printable (alphanumeric/symbol) characters — these are the ordinary

© Hak Cipta Universiti Teknologi MARA

characters such as letters, digits and punctuation that are echoed to the display and stored as character codes (e.g. ASCII); pressing 'A' or '7' produces a printable character. (2) Control characters — non-printing characters generated by keys such as Enter (carriage return/line feed), Tab, Backspace, Esc and Ctrl-combinations; they do not display a glyph but cause an action or control function in the system. (Some texts also distinguish function keys, but the two standard types are printable and control characters.)

b) Three categories of input/output module: (1) Human-readable — devices that communicate with the user, e.g. keyboard, monitor/display, printer, mouse. (2) Machine-readable — devices that communicate with equipment, e.g. disk drive, magnetic tape drive, sensor/actuator, or a controller-to-controller interface. (3) Communication — devices that allow the computer to communicate with remote devices over a network, e.g. modem, network interface card (NIC), or serial communication port.

Marking points:

- *1 mark: correctly names printable/alphanumeric characters*
- *1 mark: explains printable characters with an example (letter, digit, punctuation)*
- *1 mark: correctly names control characters*
- *1 mark: explains control characters with an example (Enter, Tab, Backspace, Esc, Ctrl-combination)*
- *1 mark: names human-readable category*
- *1 mark: gives an appropriate human-readable example (keyboard, monitor, printer, mouse)*
- *1 mark: names machine-readable category*
- *1 mark: gives an appropriate machine-readable example (disk drive, tape drive, sensor/actuator)*
- *1 mark: names communication category*
- *1 mark: gives an appropriate communication example (modem, NIC, serial port)*

Model confidence: high.

Model note: Unofficial model answer; not an official UTM marking scheme. The question text does not supply a figure or table, so the answer is based on standard CSC429 I/O-system terminology. Part (a) accepts printable vs control characters as the two types; if the lecturer's notes use a different pairing (e.g. alphanumeric vs function keys), the same explanation structure applies.

QUESTION 10 (10 marks)

Source: CSC429-2025-02, page [4]

Unofficial model answer:

a) A cache is a small, fast memory that holds recently or frequently used instructions and data so that the processor can access them with much lower latency than main memory. It sits between the CPU and main memory (RAM) and exploits locality of reference (temporal and spatial) to reduce the average memory access time.

b) Without a cache, every instruction fetch and data access would have to go to main memory. Because main memory is much slower than the CPU (often tens to hundreds of cycles per access), the processor would stall frequently, drastically reducing effective throughput and increasing average memory access time. The system would also place heavier demand on the memory bus, causing more contention and further slowdown. Overall performance would degrade significantly, and the CPU would be underutilized.

c) L1 cache is faster than L2 cache. L1 is the first-level cache, located closest to the CPU core (often on the same die and sometimes integrated into the pipeline), and is built with very fast SRAM and a small capacity (typically a few KB to tens of KB). Its small size and proximity allow access times of a few CPU cycles. L2 is the second-level cache, larger (typically hundreds of KB to several MB) and farther from the core, so it has longer access latency (tens of cycles). The trade-off is that L1 is small and expensive per bit, while L2 is larger but slower; therefore L1 provides faster speed.

d) Two write policies for cache consistency:
1. Write-through: every write to the cache is also written immediately to main memory. This keeps main memory always up to date and ensures consistency, but it generates more memory traffic and can be slower.

© Hak Cipta Universiti Teknologi MARA

2. Write-back (write-behind): writes are made only to the cache; the modified cache block is marked dirty and is written back to main memory only when it is evicted or replaced. This reduces memory traffic and is faster, but main memory can be temporarily out of date, so consistency requires careful handling (e.g., cache coherence mechanisms in multiprocessors).

Marking points:

• 1 mark: Definition of cache as small, fast memory between CPU and main memory holding recently/frequently used data/instructions.

• 1 mark: Mention of locality of reference or reduction of average memory access time.

• 2 marks: Explanation that without cache, every access goes to slow main memory, causing CPU stalls and much longer access times.

• 1 mark: Additional consequence such as increased bus traffic/contention or reduced overall performance.

• 2 marks: Identification of L1 as faster than L2.

• 1 mark: Justification based on L1 being closer to CPU/smaller/faster SRAM (fewer cycles) while L2 is larger and farther (more cycles).

• 1 mark: Write-through policy described (write to cache and main memory simultaneously).

• 1 mark: Write-back policy described (write only to cache, mark dirty, write back on eviction).

Model confidence: high.

Model note: This is an unofficial model answer, not an official UTM marking scheme. The question does not specify a particular cache organization or replacement policy, so the answer focuses on general principles. Part (d) asks for two policies to write data to the cache to ensure consistency; write-through and write-back are the standard policies, though write-back requires additional coherence mechanisms in multiprocessor systems.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION 11 (20 marks)

Source: CSC429-2013-12, page [6]

Unofficial model answer:

Subpart (a) — One-address instructions. In a one-address machine every arithmetic instruction implicitly uses an accumulator (here Register X), and the single operand is a memory location. The equation in Figure 4 is taken to be X = (A + B) × (C − D) ÷ (E + F); the figure itself was supplied only as a cropped image, so this expression is an assumption. Program: `LOAD A` (X ← A); `ADD B` (X ← X + B = A + B); `STORE T1` (save A + B); `LOAD C` (X ← C); `SUB D` (X ← C − D); `STORE T2` (save C − D); `LOAD T1` (X ← A + B); `MUL T2` (X ← (A+B)(C−D)); `STORE T3`; `LOAD E` (X ← E); `ADD F` (X ← E + F); `STORE T4`; `LOAD T3` (X ← numerator); `DIV T4` (X ← (A+B)(C−D)/(E+F)); `STORE X` (final answer in X). Temporary memory locations T1–T4 are needed because only one operand can be addressed per instruction.

Subpart (b) — Two-address instructions. A two-address instruction has the form OP (X ← Y op Z) or OP (X ← X op Y), destroying one source. Using the register-to-register forms from Table 2: `MOVE X, A` (X ← A); `ADD X, B` (X ← A + B); `MOVE Y, C` (Y ← C); `SUB Y, D` (Y ← C − D); `MUL X, Y` (X ← (A+B)(C−D)); `MOVE Z, E` (Z ← E); `ADD Z, F` (Z ← E + F); `DIV X, Z` (X ← (A+B)(C−D)/(E+F)). The final result is already in X, so no STORE is required. Only the temporary registers X, Y and Z are used; A–F are never overwritten.

Subpart (c) — Three-address instructions. A three-address instruction specifies two sources and a distinct destination, OP (X ← Y op Z), so no intermediate results need to be saved and no source is destroyed. Program: `ADD X, A, B` (X ← A + B); `SUB Y, C, D` (Y ← C − D); `MUL X, X, Y` (X ← (A+B)(C−D)); `ADD Z, E, F` (Z ← E + F); `DIV X, X, Z` (X ← (A+B)(C−D)/(E+F)). The answer ends in X as required. This is the shortest program, illustrating that three-address format minimises instruction count at the cost of a longer instruction encoding.

Marking points:

• 2 marks: correct identification/statement of the expression to be computed from Figure 4 (e.g. X = (A+B)(C−D)/(E+F))

• 6 marks: one-address program — correct use of LOAD/STORE/ADD/SUB/MUL/DIV with accumulator X, correct temporary storage of partial results, correct order of operations, final result in X (deduct for missing STORE or wrong operand order)

• 5 marks: two-address program — correct MOVE/ADD/SUB/MUL/DIV (X←Y op Z or X←X op Y) forms, correct use of temporary registers X, Y, Z, A–F unmodified, final result in X

• 5 marks: three-address program — correct OP (X←Y op Z) form, minimal instruction sequence, A–F unmodified, final result in X

• 2 marks: correct assembly notation and explanatory comments throughout; no modification of registers A–F

Model confidence: low.

Model note: Figure 4 was supplied only as a cropped image with no readable expression; the equation X = (A+B)(C−D)/(E+F) is assumed from typical past-paper style and must be checked against the actual figure. Table 2 text is garbled (e.g. 'MOVE (X e Y)', 'ADD (X k.. X + Y)'), so instruction mnemonics/operand orders are interpreted as standard one-, two- and three-address forms. This is an unofficial model answer, not an official UTM marking scheme.

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA